Files, modules and handling errors

🎯 Today's goal

By the end of this hour you will:

- use ready-made tools from Python's **modules**: `math`, `statistics` and `random`
- **write** text to a file and **read** it back
- save and load a table with the `csv` module
- catch errors with `try` / `except`, so one bad value doesn't stop everything
- raise your own error with a clear message

🤔 Why this matters

So far all your data lived inside the code. Real data lives in **files**: a spreadsheet exported as CSV, a log of sensor readings, a list of customers. Today you'll learn to save data to a file and load it again.

Real data is also **messy**. One sensor writes `"oops"` instead of a number, a file is missing, a shop had zero visitors. Without care, one bad value crashes your whole program. `try` / `except` is like a safety net under a tightrope walker: if something goes wrong, you catch it and decide what to do.

And you don't have to build everything yourself. Python comes with a big toolbox of **modules**. Data scientists use them all day: NumPy and pandas (coming on Days 9 and 10) are modules too.

🧰 Modules: Python's toolbox

A **module** is a file full of ready-made functions and values that someone else wrote. To use one, you **import** it. Then you write the module's name, a dot, and the tool you want. The `math` module has maths tools that the basic operators don't:

```
import math

print(math.sqrt(81))      # square root
print(math.pi)            # the number pi
print(math.ceil(4.2))     # round UP to a whole number
print(math.floor(4.8))    # round DOWN to a whole number

# A real use: 130 students, 50 seats per bus. How many buses?
students = 130
seats_per_bus = 50
print(students / seats_per_bus, "buses? You can't rent part of a bus...")
print("Buses needed:", math.ceil(students / seats_per_bus))
```

```
9.0
3.141592653589793
5
4
2.6 buses? You can't rent part of a bus...
Buses needed: 3
```

If you only need one or two tools, you can import them by name with `from ... import ...`. Then you can use them without the module name in front:

```
from math import sqrt, pi

radius = 2
print("Circle area:", pi * radius ** 2)
print("Square root of 2:", sqrt(2))
```

```
Circle area: 12.566370614359172
Square root of 2: 1.4142135623730951
```

The `statistics` module calculates averages for you. Its name is long, so we give it a **nickname** with `as`. You'll see this a lot: on Day 9 you'll write `import numpy as np`.

Two kinds of "average" are worth knowing. The **mean** adds everything up and divides by how many there are. The **median** is the middle value after sorting. One extreme value pulls the mean a lot, but barely moves the median:

```
import statistics as st

# Made-up waiting times at a clinic, in minutes (one patient waited 95!)
waits = [12, 15, 9, 14, 11, 95, 13]

print("Mean:  ", round(st.mean(waits), 1))
print("Median:", st.median(waits))
```

```
Mean:   24.1
Median: 13
```

The mean says "about 24 minutes", but most patients waited 9 to 15. The median (13) describes a typical patient much better.

The `random` module makes random numbers, which is handy for creating made-up data or picking a random sample. Computers make "random" numbers from a starting number called a **seed**. Fix the seed with `random.seed(42)` and you get the **same** numbers every time you run the box, so your results can be repeated:

```
import random

random.seed(42)                                # same "random" numbers every run
print(random.randint(1, 6))                    # a dice roll: a whole number from 1 to
6
print(random.choice(["tea", "coffee", "juice"]))   # pick one item from a list
print(random.random())                         # a decimal between 0 and 1

# Made-up data: 5 test scores between 50 and 100
fake_scores = [random.randint(50, 100) for i in range(5)]
print(fake_scores)
```

```
6
tea
0.025010755222666936
[67, 65, 64, 58, 97]
```

Run it twice: the numbers stay the same. Delete the `random.seed(42)` line and they change every time.

Way to import	Then you write
`import math`	`math.sqrt(9)`
`from math import sqrt`	`sqrt(9)`
`import statistics as st`	`st.mean([1, 2, 3])`

📝 Writing a text file

`open("diary.txt", "w")` opens a file for **w**riting (it creates the file, or empties it if it already exists). The `with` line means "use this file for the indented lines below, then close it properly". `f` is just a short name for the open file.

```
with open("diary.txt", "w") as f:
    f.write("Monday: 21 degrees\n")
    f.write("Tuesday: 24 degrees\n")
    f.write("Wednesday: 19 degrees\n")

print("Saved!")
```

```
Saved!
```

`\n` means "new line". `write()` doesn't start a new line by itself, so we add `\n` at the end of each line.

Where is the file? On this page, Python runs inside your browser, so files are kept in the page's memory, not on your computer's disk. They disappear when you reload the page. On your own computer, the same code creates a real file next to your program.

📖 Reading it back

Open the file without `"w"` to **read** it. `read()` gives the whole file as one string. `readlines()` gives a list with one string per line:

```
with open("diary.txt") as f:
    text = f.read()
print(text)

with open("diary.txt") as f:
    lines = f.readlines()
print(lines)          # each line still ends with \n
```

```
Monday: 21 degrees
Tuesday: 24 degrees
Wednesday: 19 degrees

['Monday: 21 degrees\n', 'Tuesday: 24 degrees\n', 'Wednesday: 19 degrees\n']
```

Very often you go through a file one line at a time with a `for` loop. `.strip()` removes the `\n` at the end. And `"a"` opens a file to **a**dd lines at the end instead of starting from empty:

```
with open("diary.txt", "a") as f:     # "a" = append: add to the end
    f.write("Thursday: 26 degrees\n")

with open("diary.txt") as f:
    for line in f:
        day, temp = line.strip().split(": ")
        print(day.upper(), "->", temp)
```

```
MONDAY -> 21 degrees
TUESDAY -> 24 degrees
WEDNESDAY -> 19 degrees
THURSDAY -> 26 degrees
```

`.split(": ")` cuts a string into a list of pieces wherever it finds `": "`, so `"Monday: 21 degrees"` becomes `["Monday", "21 degrees"]`, and we unpack the two pieces into `day` and `temp`.

Mode	Meaning
`open(name)` or `open(name, "r")`	read (the file must exist)
`open(name, "w")`	write: create the file, or empty it first
`open(name, "a")`	append: add to the end

📊 Tables in CSV files

CSV stands for "comma-separated values". It's the simplest way to store a table: one row per line, with commas between the columns. Every spreadsheet program can open and save CSV files. The `csv` module writes and reads them for you:

```
import csv

# Made-up sales of a small café: a list of rows, the first row holds the column names
sales = [
    ["day", "drink", "cups"],
    ["Mon", "tea", 31],
    ["Mon", "coffee", 45],
    ["Tue", "tea", 28],
    ["Tue", "coffee", 52],
]

with open("cafe.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(sales)       # write all rows at once

with open("cafe.csv") as f:
    print(f.read())
```

```
day,drink,cups
Mon,tea,31
Mon,coffee,45
Tue,tea,28
Tue,coffee,52
```

`newline=""` is a small detail the `csv` module asks for, so it can handle line endings itself. Just always add it when you open a CSV file.

`csv.reader` reads the file back, one row (a list) at a time. `next(reader)` takes the next row out, which is a neat way to grab the header row before the loop:

```
total_cups = 0

with open("cafe.csv", newline="") as f:
    reader = csv.reader(f)
    header = next(reader)          # the first row: the column names
    print("Columns:", header)
    for row in reader:
        print(row)
        total_cups += int(row[2])  # everything read from a file is text, so convert!

print("Total cups:", total_cups)
```

```
Columns: ['day', 'drink', 'cups']
['Mon', 'tea', '31']
['Mon', 'coffee', '45']
['Tue', 'tea', '28']
['Tue', 'coffee', '52']
Total cups: 156
```

Look closely: the rows come back as `['Mon', 'tea', '31']`. The number `31` became the text `'31'`. A file only stores text, so you convert with `int()` or `float()` after reading.

💥 When things go wrong

Here is a mistake on purpose. Nobody visited a (made-up) shop today, so the list of visits is empty, and the average divides by zero:

💡 This example has a mistake on purpose. Run it and read the error.

```
visits = []     # made up: no visitors today
average_visit = sum(visits) / len(visits)
```

⚠️ ZeroDivisionError

You divided by zero, which maths doesn't allow.

```
Traceback (most recent call last):
  line 2, in <module>
    average_visit = sum(visits) / len(visits)
                    ~~~~~~~~~~~~^~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

An error like this is called an **exception**. Normally it stops the program. But you can **catch** it: put the risky code inside `try:`, and what to do if it fails inside `except ErrorName:`. Here, one bad

reading no longer stops the loop:

```
# Made-up thermometer readings; one is broken
readings = ["21.5", "22.0", "oops", "23.5"]
good = []

for text in readings:
    try:
        good.append(float(text))    # float("oops") raises a ValueError,
    except ValueError:                # which lands here instead of crashing
        print("Skipped a bad value:", text)

print("Good readings:", good)

try:
    print(sum(visits) / len(visits))
except ZeroDivisionError:
    print("No visits, so there is no average.")
```

```
Skipped a bad value: oops
Good readings: [21.5, 22.0, 23.5]
No visits, so there is no average.
```

Opening a file that doesn't exist is another very common error:

💡 This example has a mistake on purpose. Run it and read the error.

```
with open("sales_2025.csv") as f:
    print(f.read())
```

⚠️ **FileNotFoundError**

```
Traceback (most recent call last):
  line 1, in <module>
    with open("sales_2025.csv") as f:
         ~~~~^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 44] No such file or directory: 'sales_2025.csv'
```

A `try` can have two more parts. `else:` runs only if **nothing** went wrong. `finally:` runs **always**, error or not (good for "clean-up" work):

```
file_name = "sales_2025.csv"      # try changing this to "cafe.csv"

try:
    with open(file_name) as f:
        data = f.read()
except FileNotFoundError:
    print("No file called", file_name, "yet. Starting with empty data.")
    data = ""
else:
    print("File loaded!", len(data), "characters.")
finally:
    print("Done checking for the file.")
```

```
No file called sales_2025.csv yet. Starting with empty data.
Done checking for the file.
```

Catch only what you expect. Always name the error after `except` (`except ValueError:`). A bare `except:` catches *everything*, even your own typos, and hides bugs.

🚨 Raising your own errors

Sometimes *you* know a value is wrong, even though Python wouldn't complain: a negative price, an age of 250. You can stop with a clear message using `raise`. And `except ValueError as err` catches the error *and* keeps its message in the variable `err`, so you can print it:

```
def set_price(price):
    """Return the price rounded to 2 decimals. Refuse negative prices."""
    if price < 0:
        raise ValueError(f"A price can't be negative, got {price}")
    return round(price, 2)

print(set_price(4.499))

try:
    set_price(-3)
except ValueError as err:
    print("Problem:", err)
```

```
4.5
Problem: A price can't be negative, got -3
```

Tomorrow's expense tracker uses exactly this trick to refuse a bad expense.

⚠️ Common mistakes

- Forgetting to `import` a module before using it → `NameError: name 'math' is not defined`.
- Opening a file with `"w"` when you meant to read or add: `"w"` empties it first!
- Forgetting `\n` when writing lines: everything ends up on one long line.
- Doing maths with values read from a file without converting them: they are always text.
- Wrong indentation after `with`, `try` or `except`: the lines that belong to them must be indented.
- Using a bare `except:` that hides every error, including your own typos.

✅ Recap

- `import math`, `from math import pi`, `import statistics as st`: three ways to use a module.
- `math` (sqrt, pi, ceil, floor), `statistics` (mean, median), `random` (seed, randint, choice).
- `with open(name, "w") as f:` then `f.write(...)`; read with `f.read()`, `f.readlines()` or `for line in f`.
- `csv.writer(f).writerows(rows)` saves a table; `csv.reader(f)` reads it back as lists of text.
- `try` / `except ErrorName` catches an error; `else` runs if all went well; `finally` always runs.
- `raise ValueError("message")` stops with your own clear message.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Pizza size

A (made-up) pizza has a radius of 15 cm. Using `math.pi`, calculate its area (pi × radius^2) in a variable called `area`.

```
import math

radius = 15

# Calculate area
```

Hints:

1. "Radius squared" is `radius ** 2`.
2. `area = math.pi * radius ** 2`

2. Safe divide

Finish the function `safe_divide(a, b)`. It should return `a / b`, but if `b` is zero it must **not crash**: catch the `ZeroDivisionError` with `try` / `except` and return `None` instead.

```
def safe_divide(a, b):
    return a / b
```

Hints:

1. Put `return a / b` inside a `try:` block (indented one more step).
2. Below it, add `except ZeroDivisionError:` and, indented under it, `return None`.

3. Save and read a file

Write the three city names in `cities` to a file called `cities.txt`, **one per line**. Then read the file back into a list called `lines_read`, with the `\n` removed, so it equals `["Cairo", "Lagos", "Lima"]`.

```
cities = ["Cairo", "Lagos", "Lima"]

# 1. Write each city to cities.txt, one per line

# 2. Read the file back into lines_read (without the \n)
```

Hints:

1. Writing: `with open("cities.txt", "w") as f:` then a `for` loop with `f.write(city + "\n")`.
2. Reading: start with `lines_read = []`, then `for line in f:` and `lines_read.append(line.strip())`.
3. Remember to open the file a second time for reading: `with open("cities.txt") as f:`

4. Clean the numbers

A (made-up) form collected prices as text, and some are rubbish. Go through `raw`, convert each value with `float()` and add it to a list called `numbers`. Skip the values that can't be converted using `try` / `except ValueError`, and count them in a variable called `skipped`.

```
raw = ["12.5", "n/a", "7", "", "3.25", "free"]

numbers = []
skipped = 0

# Loop over raw here
```

Hints:

1. Start with `for text in raw:` and put `try:` inside the loop.
2. Inside `try:` write `numbers.append(float(text))`.
3. Inside `except ValueError:` write `skipped += 1`.

5. Challenge: scores in a CSV file

Save the made-up test scores to a file called `scores.csv` using `csv.writer`, with the header row `name,score` first. Then read the file back with `csv.reader`, skip the header, and calculate the average score in a variable called `average`.

Calculate the average from what you read from the file, not from the list.

```
import csv

rows = [
    ["Amira", 82],
    ["Ben", 67],
    ["Chen", 91],
    ["Dana", 74],
]

# 1. Write scores.csv: the header ["name", "score"], then the rows

# 2. Read it back and calculate average
```

Hints:

1. Writing: `writer = csv.writer(f)`, then `writer.writerow(["name", "score"])` and `writer.writerows(rows)`.
2. Reading: `reader = csv.reader(f)`, `next(reader)` to skip the header, then loop and add up `int(row[1])`.
3. Count the rows as you go, then `average = total / count`.

➡️ Tomorrow

Mini-project: an expense tracker. You'll put everything from the first week together: a list of expenses stored as dictionaries, functions that add and check expenses (raising a `ValueError` for bad ones), totals per category, a budget warning, and saving everything to a CSV file and loading it back.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Pizza size

```
import math

radius = 15

area = math.pi * radius ** 2
```

2. Safe divide

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
```

3. Save and read a file

```
cities = ["Cairo", "Lagos", "Lima"]

# 1. Write each city to cities.txt, one per line
with open("cities.txt", "w") as f:
    for city in cities:
        f.write(city + "\n")

# 2. Read the file back into lines_read (without the \n)
lines_read = []
with open("cities.txt") as f:
    for line in f:
        lines_read.append(line.strip())
```

4. Clean the numbers

```
raw = ["12.5", "n/a", "7", "", "3.25", "free"]

numbers = []
skipped = 0

for text in raw:
    try:
        numbers.append(float(text))
    except ValueError:
        skipped += 1
```

5. Challenge: scores in a CSV file

```
import csv

rows = [
    ["Amira", 82],
    ["Ben", 67],
    ["Chen", 91],
    ["Dana", 74],
]

# 1. Write scores.csv: the header ["name", "score"], then the rows
with open("scores.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "score"])
    writer.writerows(rows)

# 2. Read it back and calculate average
total = 0
count = 0
with open("scores.csv", newline="") as f:
    reader = csv.reader(f)
    next(reader)
    for row in reader:
        total += int(row[1])
        count += 1
average = total / count
```